

Bot Aquaris — Documento Funcional y Alcance del Proyecto

1. Resumen ejecutivo

Bot Aquaris es un bot de Discord diseñado para centralizar la operación del servidor del clan Aquaris. Su propósito es automatizar procesos de comunidad, apoyar la moderación, ordenar el reclutamiento de nuevos miembros y habilitar herramientas internas de gestión, incluyendo actividad, recordatorios y tienda del clan.

El proyecto busca reducir la carga manual del staff, mejorar la experiencia de los usuarios y ofrecer una estructura clara para el crecimiento del servidor. A nivel funcional, el bot combina capacidades de comunidad, soporte operativo y gestión interna del clan.

2. Objetivos del proyecto

Objetivo general

Desarrollar un bot propio para Discord que permita administrar de forma consistente los procesos clave del servidor Aquaris.

Objetivos específicos

- Automatizar la asignación de roles iniciales y la identificación visual de usuarios.
 - Mejorar la bienvenida, orientación y participación de los miembros.
 - Facilitar al staff herramientas de moderación y trazabilidad.
 - Estandarizar el proceso de reclutamiento mediante tickets.
 - Registrar y exponer métricas de actividad de la comunidad.
 - Incorporar utilidades específicas del servidor, como recordatorios personales.
 - Centralizar la operación de tienda, pedidos e inventario del clan.
-

3. Alcance funcional

El bot se compone de los siguientes módulos:

1. Gestión de usuarios y automatización
 2. Bienvenida y salida
 3. Sugerencias
 4. Boost del servidor
 5. Sistema de niveles y actividad
 6. Perfil y rankings
 7. Recordatorios privados
 8. Moderación y logs
 9. Tienda del clan
 10. Reclutamiento por tickets
-

4. Propuesta de implementación por fases

Acá es donde el documento mejora de verdad: separar lo importante de lo deseable.

Fase 1 — MVP operativo

Esta fase cubre lo mínimo necesario para que el bot aporte valor real desde el inicio:

- Autorol al entrar
- Rol Aspirante al abrir ticket de reclutamiento
- Autonickname por jerarquía
- Mensajes de bienvenida y despedida
- Sistema de sugerencias
- Moderación básica
- Logs de moderación
- Reclutamiento por ticket con flujo completo básico

Resultado esperado: el servidor ya queda mejor organizado, el staff ahorra tiempo y el proceso de entrada al clan deja de depender de manejo manual.

Fase 2 — Engagement y comunidad

- Sistema de niveles por mensajes y voz
- Métricas de actividad
- Perfil de usuario
- Rankings
- Mensajes de agradecimiento por boost
- Recordatorios privados

Resultado esperado: mayor participación y mejor visibilidad sobre actividad real de los miembros.

Fase 3 — Operación avanzada del clan

- Tienda del clan
- Gestión de productos
- Gestión de pedidos
- Gestión de inventario
- Registro de ventas
- Control de retiradas
- Posible exportación a Google Sheets

Resultado esperado: formalizar la economía interna del clan con trazabilidad y control. Esta es la parte más compleja y debería ir después, no antes.

5. Especificación funcional por módulo

5.1 Gestión de usuarios y automatización

Objetivo

Automatizar el acceso inicial al servidor, la asignación de roles base y la identificación visual según jerarquía.

Funcionalidades

- Asignar automáticamente el rol **Visitante** cuando un usuario entra por primera vez.
- Asignar automáticamente el rol **Aspirante** cuando un usuario abre un ticket de reclutamiento.
- Añadir prefijo al nickname según el rol del usuario.

Ejemplos de prefijo

- LIDER | nombre
- CO-LIDER | nombre
- AQUARIS | nombre
- STAFF | nombre

Reglas funcionales

- El sistema no debe borrar el nombre original del usuario.
- El prefijo debe actualizarse si el rol cambia.
- El staff podrá modificar manualmente el apodo sin romper la lógica del sistema.

- Debe evitarse sobrecribir apodos de manera conflictiva.

Criterios de aceptación

- Cuando un usuario nuevo entra al servidor, recibe el rol Visitante.
 - Cuando un usuario inicia reclutamiento, recibe el rol Aspirante.
 - Cuando un miembro cambia de rol, su prefijo se actualiza correctamente.
 - El cambio manual de apodo por parte del staff no genera errores ni duplicados.
-

5.2 Bienvenida y despedida

Objetivo

Dar contexto al usuario desde su entrada al servidor y mantener visibilidad sobre ingresos y salidas.

Funcionalidades

- Mensaje embed de bienvenida en canal configurado.
- Mensaje embed de despedida cuando alguien abandona el servidor.

Contenido mínimo de bienvenida

- Mención al usuario
- Mensaje de bienvenida
- Enlaces a canales importantes

Criterios de aceptación

- Todo nuevo usuario recibe una bienvenida visible en el canal definido.
 - Todo usuario que salga del servidor genera un mensaje de despedida.
 - El embed incluye los accesos básicos para orientarse.
-

5.3 Sistema de sugerencias

Objetivo

Permitir que la comunidad proponga mejoras de forma ordenada y medible.

Funcionalidades

- Comando `/suggest`
- Publicación automática de la sugerencia en canal designado
- Reacciones para votar
- Estado de sugerencia gestionable por staff:
 - aceptada
 - rechazada
 - en revisión

Criterios de aceptación

- Un usuario puede enviar una sugerencia sin intervención del staff.
 - La sugerencia aparece formateada como embed.
 - El staff puede marcar el estado posterior de esa sugerencia.
-

5.4 Boost del servidor

Objetivo

Reconocer públicamente a los usuarios que apoyan el servidor con boost.

Funcionalidades

- Mensaje automático de agradecimiento
- Mención al booster
- Posibilidad futura de rol especial Booster

Nota de alcance

Este módulo puede incluirse en fase 2 porque no es crítico para la operación base.

5.5 Sistema de niveles y actividad

Objetivo

Medir y recompensar la participación en el servidor.

Funcionalidades

- Ganancia de XP por mensajes
- Ganancia de XP por tiempo en voz
- Subida automática de nivel
- Estadísticas generales de actividad
- Leaderboard de actividad

Reglas a definir en desarrollo

- Cooldown de XP por mensaje
- Peso relativo entre mensajes y voz
- Exclusiones de canales spam o bots
- Prevención de abuso/farmeo

Criterios de aceptación

- La actividad del usuario incrementa su XP según reglas configuradas.
 - El sistema calcula el nivel automáticamente.
 - Los datos quedan disponibles para perfiles y rankings.
-

5.6 Perfil y rankings

Objetivo

Mostrar de forma clara el historial básico y la actividad del usuario dentro del servidor.

Funcionalidades

- `/perfil`
- `/perfil [usuario]` para staff
- `/top mensajes`
- `/top voz`

Datos del perfil

- nombre del usuario
- rol actual
- tiempo en el servidor
- tiempo en el clan si aplica
- número de mensajes
- tiempo en voz
- nivel de actividad

Criterios de aceptación

- Cualquier usuario puede consultar su propio perfil.
 - El staff puede consultar el de otros usuarios.
 - El sistema muestra rankings consistentes de mensajes y voz.
-

5.7 Recordatorios privados

Objetivo

Ayudar a los jugadores a gestionar cooldowns y actividades recurrentes sin saturar los canales del servidor.

Funcionalidades

- `/remind crear`
- `/remind reset`
- `/remind ver`
- `/remind eliminar`

Casos de uso

- Kits de reutilización periódica
- Dungeons diarias
- Eventos temporales
- Otras actividades con tiempos de espera

Requisitos funcionales

- Las respuestas deben ser privadas o efímeras.
- Debe permitirse primer aviso y recurrencia automática.
- El usuario debe poder reiniciar el contador manualmente.

Criterios de aceptación

- Un usuario puede crear un recordatorio con nombre, primer aviso y ciclo.
 - El bot notifica cuando el tiempo se cumple.
 - El usuario puede consultar y resetear sus recordatorios.
-

5.8 Moderación y logs

Objetivo

Facilitar el trabajo del staff y mantener trazabilidad de sanciones y acciones administrativas.

Funcionalidades principales

- `/ban`
- `/unban`
- `/tempban`
- `/kick`
- `/mute`
- `/tempmute`
- `/unmute`
- `/timeout`
- `/warn`
- `/warnings`
- `/clear`
- `/filter` añadir
- `/filter` eliminar
- `/filter` lista
- `/logs` usuario
- `/reason id_sancion motivo`

Capacidades automáticas

- Detección de spam de mensajes
- Detección de spam de menciones
- Detección de spam de enlaces
- Detección de spam de emojis
- Aplicación automática de advertencia, borrado de mensajes o mute temporal según reglas configuradas

Requisitos mínimos de auditoría

Toda acción debe registrar:

- usuario afectado
- moderador
- motivo
- duración si aplica

- fecha y hora

Recomendación profesional

Acá te falta definir umbrales concretos. Por ejemplo:

- 5 mensajes en 8 segundos
 - 4 menciones en 10 segundos
 - 3 enlaces en 20 segundos
- Eso ya se define en diseño técnico, pero debe quedar previsto desde producto.
-

5.9 Tienda del clan

Objetivo

Centralizar la venta de materiales y servicios mediante un flujo ordenado basado en tickets.

Flujo funcional

1. El usuario consulta productos o precios.
2. Abre un ticket de pedido.
3. El bot envía el pedido al canal privado de staff.
4. El staff acepta o rechaza.
5. Si se acepta, se crea canal temporal.
6. Al completar la entrega, el ticket se cierra y la venta se registra.

Comandos de usuario

- `/tienda ver`
- `/tienda precio material`
- `/pedido crear`
- `/pedido estado ID`
- `/top compradores`
- `/ayuda tienda`

Comandos de staff

- `/tienda añadir material precio`
- `/tienda eliminar material`
- `/tienda precio material nuevo_preio`

- `/tienda activar material`
- `/tienda desactivar material`
- `/stock ver`
- `/stock actualizar material cantidad`
- `/stock sumar material cantidad`
- `/stock restar material cantidad`
- `/stock bajo`
- `/pedido aceptar ID`
- `/pedido rechazar ID motivo`
- `/pedido cerrar ID`
- `/pedido cancelar ID`
- `/pedido lista`
- `/ventas hoy`
- `/ventas semana`
- `/historial ventas material`
- `/retirar material cantidad motivo`

Riesgos funcionales

Este módulo requiere mucha más precisión que los demás:

- control de concurrencia de stock
- reservas temporales
- cancelaciones
- reversión automática
- integridad del historial

Recomendación

No lo empieces sin modelo de datos claro. Esta parte necesita diseño técnico serio.

5.10 Reclutamiento por tickets

Objetivo

Ordenar el ingreso de nuevos miembros y documentar cada solicitud desde el primer contacto hasta la aceptación o rechazo.

Flujo funcional

1. Usuario abre ticket en canal de reclutamiento.
2. Selecciona tipo de rol al que aplica.
3. Responde formulario.
4. Bot crea ticket y asigna rol Aspirante.
5. Staff revisa solicitud.
6. Staff acepta o rechaza.
7. Si acepta, se coordina entrevista.
8. Si entrevista es satisfactoria, se asigna rol Aquaris.

Tipos de rol de aplicación

- Builder
- Técnico / Redstone
- PvP
- Farmer
- Otro

Preguntas recomendadas

- Edad
- Desde cuándo juegas Minecraft
- Experiencia en servidores o SMP
- Qué rol quieres desempeñar
- Qué puedes aportar
- Disponibilidad aproximada

Extras previstos

- Adjuntar imágenes de construcciones, diseños o ejemplos del trabajo del aspirante.

Reglas del flujo

- Si se rechaza, el bot notifica el motivo.
- Si se acepta, el bot abre conversación temporal con staff.
- El rol Aspirante habilita acceso a espacios de entrevista.
- Si se aprueba la entrevista, el staff asigna Aquaris y el usuario entra al clan.

Criterios de aceptación

- Toda solicitud queda registrada en ticket.
 - El staff puede revisar, aceptar o rechazar desde flujo definido.
 - El usuario recibe retroalimentación automática.
 - El ingreso final al clan requiere intervención explícita del staff.
-

6. Requisitos no funcionales

Seguridad y permisos

- Los comandos administrativos solo deben estar disponibles para roles autorizados.
- Los tickets privados deben respetar permisos por categoría/canal.
- El historial de moderación no debe ser visible para usuarios comunes.

Trazabilidad

- Toda acción sensible debe quedar registrada.
- Debe existir identificador único para pedidos, tickets y sanciones.

Mantenibilidad

- Arquitectura modular por dominios: comunidad, moderación, reclutamiento, tienda, utilidades.
- Configuración por variables o archivo para canales, roles, prefijos, límites y mensajes.

Experiencia de usuario

- Preferir slash commands.
- Respuestas efímeras cuando aplique.
- Embeds consistentes para mensajes del bot.
- Textos claros, cortos y fáciles de entender.

Escalabilidad

- La tienda y estadísticas deben persistir en base de datos.
 - El sistema debe soportar crecimiento en usuarios, pedidos y registros sin depender de memoria local.
-

7. Priorización recomendada

Prioridad alta

- Autorol
- Autonickname
- Bienvenida/despedita
- Reclutamiento
- Moderación

- Logs

Prioridad media

- Sugerencias
- Boost
- Perfiles
- Rankings
- Niveles

Prioridad baja o avanzada

- Recordatorios privados
 - Tienda
 - Inventario
 - Ventas
 - Exportación a Google Sheets
 - Eventos de tienda
 - Merch
 - Calculadora
-

8. Riesgos del proyecto

- Querer lanzar todos los módulos al mismo tiempo.
 - Falta de definición de reglas exactas en moderación automática.
 - Conflictos entre nickname automático y cambios manuales.
 - Complejidad operativa de tienda e inventario.
 - Crecer en features sin una base de permisos y persistencia sólida.
-

9. Recomendación final

Mi recomendación profesional es esta:

no intentes construir el bot completo de una sola vez.

Tu idea está buena, pero el valor real aparece cuando sacas una primera versión estable. El mejor camino es lanzar primero el núcleo del servidor: roles, reclutamiento, bienvenida, moderación y logs. Después metes engagement. La tienda la dejas para cuando ya tengas una base sólida.